3.3 多通道策略与群组管理

> **生成模型**：gpt-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~18k tokens，输出 ~4k tokens（本节）

把一个通道接通，和把多个通道“管明白”，完全不是一回事。

很多人第一次玩 OpenClaw，会有一种很自然的冲动：Telegram 跑通了，Discord 也接上了，Slack 也能回了，那是不是只要把 `enabled: true` 多写几个，就等于搞定多通道了？严格说，不算。

因为真正麻烦的，不是“能不能同时开”，而是：

- 不同通道要不要共用同一种响应规则
- 私聊和群聊是不是该走同一套策略
- 哪些群能进，哪些群能看但不回
- 机器人什么时候必须被 `@`，什么时候可以主动回
- 同一个 OpenClaw，需不需要给不同通道分派不同 Agent

这一节就是解决这些问题。你会发现，多通道管理本质上不是“多开几个接口”，而是在做一套消息秩序设计。

3.3.1 多通道不是越多越强，而是越多越需要边界

先说一个经验判断：**通道越多，越不能靠默认设置硬顶。**

原因很简单。不同平台的聊天气质差很多：

- Telegram 常常是个人私聊、小群、机器人比较自然的环境
- Discord 偏社区和协作，频道结构清楚，命令和提及很多
- Slack 偏工作流，线程和频道秩序很重要

如果你拿同一套“看到消息就回”的策略硬套三边，结果通常不是“统一”，而是“同时失控”。

所以多通道第一原则不是扩容，而是分层：

- 哪些通道是主入口
- 哪些通道只是补充入口

- 哪些通道只允许私聊
- 哪些通道可以进群，但必须严格门控

3.3.2 OpenClaw 同时跑多个通道，配置上长什么样

从配置结构上看，多通道其实很直观：在 `channels` 下面把多个平台都打开就行。

例如：

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTelegramToken",
      dmPolicy: "pairing",
      mentionGating: true,
      allowlist: {
        users: ["your_telegram_username"],
        chats: [-1001234567890],
      },
    },

    discord: {
      enabled: true,
      token: "YOUR_DISCORD_BOT_TOKEN",
      mentionGating: true,
      allowlist: {
        guilds: ["123456789012345678"],
        channels: ["234567890123456789"],
      },
    },

    slack: {
      enabled: true,
      appToken: "xapp-1-A0000000000-0000000000-abcdefghijklmnop",
      botToken: "你的-slack-bot-token",
      mentionGating: true,
      allowlist: {
        channels: ["C0123456789"],
      },
      threads: {
        enabled: true,
      },
    },
  },
}
```

你会发现，虽然每个平台字段不一样，但思路很像：

- 平台凭证
- 是否启用

- 私聊 / 群聊策略
- allowlist 限定范围
- mention 或 command 门控

这就是多通道配置的核心统一性。OpenClaw 不是把所有平台揉成一个完全一样的配置，而是把“管理思路”统一了。

3.3.3 先搞懂两个特别关键的门：mention gating 和 command gating

如果你只记这一节两件事，我希望就是这两个 gating。

Mention Gating

它的意思前面已经提过：在群聊里，只有别人明确 `@机器人` 时，机器人才响应。

这个策略适合大多数群聊，因为它天然减少误触发。群里十个人闲聊时，Bot 不会自己跳出来接每一句。

Command Gating

这个门更严格。意思是：只有符合命令格式的消息，它才处理。比如：

```
/ask 总结一下刚才的讨论
!todo 把这周任务列出来
```

这种方式在 Discord 和某些偏工具型频道里特别有用，因为它能把“聊天”和“调用机器人”切得很开。

两个门怎么选

- 聊天感更强的群：优先 mention gating
- 工具感更强的群：优先 command gating
- 噪音很高的大群：两个都上

最稳的一般不是“全开”，而是“根据群的用途选门”。

3.3.4 群聊为什么默认应该比私聊更保守

私聊和群聊别用一套脑回路。

私聊里，用户通常是明确来找机器人的；群聊里，机器人只是旁边站着的一个参与者。这个角色差异会直接决定你的策略。

我建议你默认这么想：

- 私聊：可以相对开放，但最好先配对或授权
- 群聊：默认保守，先限制再放开

原因不只是“怕打扰别人”，更因为群聊里有三类额外风险：

1. 噪音高，机器人容易被无关对话触发
2. 语境复杂，容易误读上一句是谁对谁说的
3. 一旦回复失误，影响范围比私聊大得多

所以一个成熟点的做法，往往是：

- 私聊允许连续对话
- 群聊必须 `@` 或命令触发
- 只有特定群进入 allowlist

3.3.5 Allowlist 不是高级功能，而是多通道的基本盘

很多人觉得 allowlist 是“以后安全加固再说”的东西，我不太同意。对多通道来说，它应该是基础设施。

前面解释过，allowlist 就是“允许通过的名单”。它可以按不同维度去限定：

- 用户
- 群组 / 频道
- 服务器 / 工作区
- 某些账号身份

为什么它这么重要？因为一旦你同时开多个平台，你很快就会遇到这些情况：

- 这个 Telegram 小群想用，另一个不想用
- Discord 里只有研发频道要接，闲聊频道不要
- Slack 只想在 `#ops-ai` 和 `#daily-sync` 里开

如果没有 allowlist，你只能靠“别把机器人拉进去”这种人工约束。这个约束极不可靠。

3.3.6 一个更像生产环境的多通道配置思路

下面给一个更偏策略化的例子。重点不是字段绝对精确，而是你要看懂“同一个 OpenClaw，不同通道可以有不同性格”。

```
{
  channels: {
    telegram: {
      enabled: true,
      botToken: "123456789:AAExampleTelegramToken",
      dmPolicy: "pairing",
      mentionGating: false,
      allowlist: {
        users: ["your_telegram_username"],
      },
    },

    discord: {
      enabled: true,
      token: "YOUR_DISCORD_BOT_TOKEN",
      mentionGating: true,
      commandGating: true,
      allowlist: {
        guilds: ["123456789012345678"],
        channels: ["234567890123456789", "234567890123456790"],
      },
    },

    slack: {
      enabled: true,
      appToken: "xapp-1-A0000000000-0000000000-abcdefghijklmnop",
      botToken: "你的-slack-bot-token",
      mentionGating: true,
      allowlist: {
        channels: ["C0123456789", "C9876543210"],
      },
      threads: {
        enabled: true,
      },
    },
  },
}
```

这套策略表达的是：

- Telegram 当作“我自己的私聊入口”，所以主要放开个人账号
- Discord 当作“团队频道工具”，所以必须 `@` 或命令触发
- Slack 当作“工作区协作入口”，所以只在指定频道、指定线程习惯下工作

你看，多通道不是所有门一样宽，而是每个平台按场景调门。

3.3.7 群组管理里最容易忽略的一件事：别让机器人角色模糊

有些 Bot 为什么让人越用越烦？不是因为它不聪明，而是因为它在群里的角色不清楚。

群里机器人常见有三种角色：

被动助手

别人叫它，它才回答。这个最稳，最适合大多数 OpenClaw 落地场景。

命令工具

它主要执行明确命令，比如重置会话、整理纪要、生成待办。这种适合 Discord、Slack 这类协作平台。

半主动协作者

它会在某些条件下自己介入，比如 thread 中持续跟进，或者对特定关键词做回应。这个最难控，建议在你已经把前两种玩顺以后再碰。

角色一旦不清楚，结果通常就是：用户不知道什么时候该叫它，它也不知道什么时候该闭嘴。

3.3.8 每个通道可以有自己的设置，不要强求完全一致

很多工程师第一次做多通道，会有一种配置洁癖：想把所有平台弄成一模一样。其实没必要。

不同通道本来就适合不同交互风格。

例如：

- Telegram 更适合长一点的私聊往返
- Discord 更适合频道内 `@` 提问和命令型操作
- Slack 更适合线程延续和工作流回复

所以“按通道定策略”不是妥协，而是正常做法。你真正要追求的一致性，不是表面字段一致，而是用户预期一致：

- 想私聊它时，它像一个可靠助手
- 想在群里叫它时，它不会乱插话
- 想限定使用范围时，它真的守边界

3.3.8.1 可以按通道单独调哪些东西

很多人第一次看到“channel-specific settings”会觉得很玄，其实就是一句话：**每个平台可以单独调参数。**

常见能单独调的东西包括：

- 是否允许私聊直接进入主对话
- 群里是否必须 `@`
- 是否只认命令前缀
- 单条消息分段长度
- 是否优先在线程里回复
- 是否限制在某些用户、某些群、某些频道里工作

比如你可能会这样想：

- Telegram 私聊像手机短信，适合自然对话，所以门可以稍微宽一点
- Discord 群聊人多嘴杂，最好必须 `@` 或命令触发
- Slack 是工作协作场景，最好把回复尽量收进线程里，别把主频道刷满

这就是 channel-specific settings 的价值。不是为了把配置写复杂，而是为了让机器人更像“懂场合的人”。

3.3.8.2 群组可以按用途分三层管理

如果你的机器人开始进入多个群，建议别一个群一个群随缘设置，最好先做分类。一个很实用的分法是三层：

第一层：核心工作群

这类群是你真的希望机器人长期参与的，比如研发群、值班群、项目推进群。这里可以给它更清晰的权限和更稳定的上下文策略。

建议：

- 进入 allowlist
- 开启 mention gating
- 视情况保留 command gating
- 明确它在群里的角色，是纪要助手、技术问答，还是值班辅助

第二层：实验群

这类群是你用来试配置、试行为、试提示词的。它们很重要，因为很多事故都应该死在实验群里，而不是正式群里。

建议：

- 单独放进 allowlist
- 允许更激进一点的尝试
- 新功能先在这里跑

第三层：观察群或禁止群

有些群你可能只是想先放着看，或者根本不希望它工作。那最简单的办法就是别进 allowlist，让它天然不处理。

这个分层做久了，你会发现排错也轻松很多。因为一旦某个群行为异常，你先看它属于哪一层，就大概知道应该用哪种标准判断。

3.3.9 Per-channel agent routing：按通道分配不同 Agent 的基础思路

这是很多人接到第三个通道以后才会开始想的事：**同一个 OpenClaw，有没有必要让不同通道走不同 Agent?**

答案通常是：有，而且很常见。

比如：

- Telegram 私聊走“个人助理 Agent”
- Discord 技术群走“技术支持 Agent”
- Slack 值班频道走“运维值班 Agent”

这就叫按通道路由 Agent。你可以理解成：通道不仅决定消息从哪进来，也能影响消息最后交给谁处理。

这件事的好处非常大：

- 不同平台能有不同语气和职责
- 上下文不会全混在一起
- 群里的工作指令不会污染你的个人私聊记忆

这一节我们不展开源码，只先建立一个最重要的直觉：**通道是入口，Agent 是大脑，入口不同，大脑未必应该完全一样。**

3.3.10 一个基础的多 Agent 配置示意

下面给一个概念性例子，帮助你理解“每个通道走不同 Agent”这件事：

```
{
  agents: {
    defaults: {
      model: "anthropic/claude-sonnet-4-5",
    },
    list: [
      { id: "personal" },
      { id: "team-helper" },
      { id: "ops-bot" },
    ],
  },

  routing: {
    byChannel: {
      telegram: "personal",
      discord: "team-helper",
      slack: "ops-bot",
    },
  },
}
```

你不用纠结这里的字段在你当前版本里是不是逐字可用。重点是理解策略：

- Telegram 来的消息，交给偏个人化的 Agent
- Discord 来的消息，交给偏群协作的 Agent

- Slack 来的消息，交给偏流程和值班的 Agent

这就是 per-channel agent routing 的基本想法。

3.3.11 多通道上线前，建议你做一轮“边界测试”

很多人测试 Bot，只测“会不会回”。其实多通道更该测“该不该回的时候会不会闭嘴”。

建议你至少做这几组测试：

测试 1：允许的私聊是否正常

在 allowlist 内的账号上测试私聊，确认能正常连续对话。

测试 2：不允许的对象是否被拦住

用一个不在 allowlist 的账号或频道测试，确认它不会错误放行。

测试 3：群里不 @ 是否沉默

如果开了 mention gating，那就故意发一条普通消息，确认机器人不跳出来。

测试 4：命令前缀是否生效

如果开了 command gating，就测同一句话在“带命令”和“不带命令”时是否表现不同。

测试 5：不同通道是否走了不同角色

如果你做了 per-channel agent routing，可以给不同平台发同类问题，看它们的语气、职责是否符合预期。

3.3.11.1 再补一组特别实用的群组管理检查清单

正式把机器人拉进多个群之前，我建议你拿这份清单过一遍：

1. 这个群到底需不需要机器人，还是只是“顺手也加一下”
2. 它在这个群里是被动助手，还是命令工具
3. 不 `@` 它时，它是不是应该完全沉默
4. 这个群是否真的进了 allowlist
5. 群成员是否知道怎么正确触发它

6. 如果它答错或刷屏，谁来第一时间处理

你会发现，这份清单大半都不是技术问题，而是运营问题。多通道系统到后面一定会碰到这个层面。

3.3.12 你真正要设计的，不是接口，而是秩序

写到这里，你应该已经能感觉到：第 3 章虽然在讲“接通道”，但真正难的地方不是复制 token，也不是把 `enabled: true` 填进去。

真正难的是你要想清楚：

- 谁能找到这个机器人
- 谁能触发它
- 在什么场景下它该说话
- 在什么场景下它必须闭嘴
- 不同平台上的它，是不是同一个角色

这些决定看起来像配置，其实是在设计系统边界。边界设计得好，机器人会像一个懂分寸的助手；边界设计得差，它就会像一个在每个平台里乱撞的噪音源。

本节小结

1. 多通道管理的重点不是“同时开几个平台”，而是为不同平台设计不同边界和行为规则。
2. 群聊默认应当比私聊更保守， `mention gating` 和 `command gating` 是最关键的两道门。
3. allowlist 是多通道环境里的基础设施，它决定机器人到底能在哪些人、哪些群、哪些频道里工作。
4. 不同通道没必要强行统一交互风格，Telegram、Discord、Slack 本来就适合不同使用方式。
5. per-channel agent routing 的核心思想是“入口不同，大脑可以不同”，这会让多通道系统更清晰，也更稳。